ESPECIFICACIÓN DE REQUISITOS

Guía IEEE para la Especificación de Requisitos Software IEEE-STD-830-1998

Fundamentos de Ingeniería del Software

©2021

Universidad Politécnica de Madrid
E.T.S. de Ingeniería en Sistemas Informáticos
Departamento de Sistemas Informáticos

1 INTRODUCCIÓN

Una **Especificación de Requisitos Software** (**ERS**) (*SRS: Software Requirements Specification*) es un documento técnico que permite definir los requisitos obtenidos en las fases de elicitación y análisis. Este documento se convierte en una herramienta de comunicación entre el cliente y los ingenieros software.

1.1 Objetivos

La ERS tiene como principales objetivos:

- Ayudar a los clientes a describir claramente lo que se desea obtener de un determinado sistema o producto software
- Ayuda a los ingenieros de software a entender qué quiere exactamente el cliente

1.2 Ventajas

La realización de una buena ERS proporciona las siguientes ventajas:

- Contrato entre el cliente y los ingenieros de software
- Reducción del esfuerzo de la construcción, operación y mantenimiento del software
- Base para la estimación de costes y planificación
- Punto de referencia para procesos de verificación y validación
- Mecanismo facilitador para la transferencia de productos software
- Base para futuras mejoras

1.3 Las consideraciones para producir un buen ERS.

1.3.1 Naturaleza del ERS

La ERS es una especificación para un producto del software, programa, o conjunto de programas concretos, que realizan ciertas funciones en un entorno específico. La ERS será escrita por uno o más representantes de los ingenieros software, por uno o más representantes del cliente, o por ambos.

Debe tratar las siguientes cuestiones:

a) **Funcionalidad**: *¿Qué es lo que debe hacer el software?*

b) **Interfaces Externas**: *¿Cómo interactúa el software con los usuarios, el hardware del sistema, o con otro software o hardware?*

c) **Rendimiento**: *¿Cuál es la velocidad, tiempo de respuesta, tiempo recuperación, etc. de determinadas funciones del software?*

d) **Atributos**: *¿Qué debe cumplir el producto/sistema respecto a portabilidad, corrección, seguridad, mantenimiento, etc.?*

e) **Restricciones de diseño e implementación impuestas**: *¿Hay alguna restricción con respecto a estándares a seguir, lenguaje de implementación, limitación de recursos, políticas para garantizar la integridad de las bases de datos, etc.?*

1.3.2 Entorno de la ERS

Es importante tener en cuenta el papel que juega una ERS dentro de un proyecto global. El software a desarrollar puede contener toda la funcionalidad del proyecto o puede ser parte de un sistema más grande. En el último caso, es importante especificar las interfaces entre el sistema y la porción de software que se está desarrollando.

La ERS tiene un rol específico en el proceso de desarrollo de software, por lo que no se deben traspasar sus límites. Esto significa que una ERS:

a) Debe definir todos los requisitos del software correctamente.

b) No debe describir detalles de diseño o implementación.

c) No debe imponer las restricciones adicionales al software. Estas se especifican en otros documentos (planificación y gestión del proyecto, planes de calidad, etc.).

Una ERS limita el conjunto de soluciones de diseño válidas, pero no especifica ningún diseño en particular.

1.3.3 Características de una buena ERS.

Una ERS debe ser:

a) Correcta: Una ERS es correcta si, y solo si, cada requisito declarado se encuentra en el software.

b) No ambigua: Una ERS no es ambigua si, y solo si, cada requisito declarado tiene solo una interpretación. Ejemplo de ambigüedad: Todos los clientes tienen el mismo campo de control. Esta frase en lenguaje natural se puede interpretar como: (1) ¿todos tienen el mismo valor en el campo de control?, (2) ¿todos los campos de control tienen el mismo formato?, o (3) ¿un campo de control se usa para todos los clientes?

c) Completa: Una ERS es completa si, y solo si, incluye los elementos siguientes:

i. Inclusión de todos los requisitos significativos.
ii. Definición de respuestas del software a todas las posibles entradas del sistema, tanto válidas, como inválidas, en todas las posibles situaciones.
iii. Etiquetado de figuras, tablas, diagramas, etc., así como la definición de términos y unidades de medidas utilizados.

Evitar en la medida de lo posible el uso de las frases: *A determinar* o *Por definir.*

d) Consistente: Una ERS es consistente si no recoge requisitos en conflicto mutuo. Hay tres tipos de posibles conflictos:

i. Características especificadas sobre la definición del problema que entren en conflicto.
ii. Conflictos lógicos o temporales entre dos acciones especificadas. Por ejemplo: A siempre después de B, vs. *A y B simultáneamente*.
iii. Dos o más requisitos pueden describir el mismo objeto del mundo real, pero usando diferentes términos.

e) **Clasificada por importancia y/o estabilidad**: Una ERS está clasificada por importancia y/o estabilidad si cada requisito tiene un identificador para indicar la importancia o estabilidad de ese requisito en particular.

No todos los requisitos que se detallan de un determinado producto del software son igual de importantes. Algunos de ellos son esenciales, mientras que otros son deseables. Es por ello, que se deben establecer estas diferencias de forma clara y explícita.

f) Verificable: Una ERS es verificable si, y solo si, cada uno de los requisitos que contiene es verificable. Un requisito es verificable si existe algún proceso no muy costoso por el que una persona o máquina puede comprobar que el software satisface el requisito. Ejemplos de requisitos no verificables: (1) El producto debería funcionar bien, o (2) El producto debería tener una buena interfaz de usuario. Ejemplo de requisito verificable: La salida se suministra dentro de los 20 segundos siguientes al evento E el 60% de la veces, y en los 30 segundos siguientes en el 100%.

g) Modificable: Una ERS es modificable si, y solo si, su estructura y estilo son tales que puede realizar cualquier cambio en los requisitos de manera fácil, completa y consistente. Para ello es necesario:

i. Organización coherente y fácil de usar (índice, referencias cruzadas, etc.)
ii. No ser redundante.
iii. Expresar cada requisito separadamente, no mezclado con otros.

h) Explorable (Trazabilidad): Una ERS es explorable si el origen de cada uno de sus requisitos está claro tanto *hacia atrás* como *hacia adelante*.

i. Trazabilidad hacia atrás: Cada requisito referencia explícitamente su fuente en documentos previos.
ii. Trazabilidad hacia adelante: Cada requisito en la ERS que tiene un único nombre o número de la referencia. Esta trazabilidad es importante para saber los cambios de los requisitos ante cambios en el diseño o en el código durante las fases de desarrollo o mantenimiento.

1.3.4 Preparación conjunta de la ERS

Para hacer una buena ERS, el ingeniero de software y el cliente deben realizar la ERS de forma conjunta, para que cada uno de la visión que el otro no tiene. De esta forma, se convertirá en el acuerdo entre el ingeniero de software y el cliente sobre qué debe hacer el producto/sistema software.

1.3.5 Evolución de la ERS

La ERS debe evolucionar conforme lo hace el proceso de desarrollo de software debido a que puede ser imposible especificar ciertos detalles al comienzo del proyecto o que conforme el producto evolucione pueden aparecer deficiencias, incorrecciones, etc. que no hubieran sido detectadas en los requisitos previamente.

Deberá establecerse un sistema de control de cambio en los requisitos.

1.3.6 Prototipado

Los prototipos frecuentemente se usan durante la extracción de requisitos porque son muy útiles para identificar requisitos de usuarios y el alcance de la ERS, y así sufrir menos cambios durante las siguientes etapas.

1.3.7 Inclusión del diseño

La ERS debería definir restricciones de diseño, limitando las alternativas de diseño, pero no describir un diseño concreto. La ERS no debe especificar elementos de diseño como: (1) partición del software en módulos, (2) asignar funcionalidad a módulos, (3) describir el flujo de información o control entre módulos, o (4) seleccionar estructuras de datos.

1.3.8 Inclusión de requisitos del proyecto

La ERS tiene como objetivo describir las necesidades del producto software, y no del proceso que lo construye. Los requisitos del proyecto no se deben incluir en un ERS. Algunos ejemplos de estos requisitos de proyecto son: coste, planificación de entregas, informes de procedimientos, métodos de desarrollo software, garantía de la calidad, criterios de validación y verificación, y procedimientos de aceptación.

2 PLANTILLA DE ESPECIFICACIÓN DE REQUISITOS

A continuación, se muestra la plantilla que sirve como modelo para una ERS según el estándar IEEE-STD-830-1998.

Tabla de Contenidos

1. Introducción
    - 1.1 Propósito
    - 1.2 Alcance
    - 1.3 Definiciones, siglas, y abreviaturas
    - 1.4 Referencias
    - 1.5 Visión global
2. Descripción general
    - 2.1 Perspectiva del producto
    - 2.2 Funciones del producto
    - 2.3 Características del usuario
    - 2.4 Restricciones
    - 2.5 Supuestos y dependencias
    - 2.6 Requisitos pospuestos
3. Requisitos específicos

Apéndices

Índice

La sección 3 de requisitos específicos es la más extensa del estándar e importante, puesto que contiene los requisitos software. El estándar propone varias posibilidades para especificarla. En particular en la asignatura vamos a adoptar la que se presenta a continuación.

1. Introducción
    - 1.1 Propósito
    - 1.2 Alcance

1.3 Definiciones, siglas, y abreviaturas
1.4 Referencias
1.5 Visión global
2. Descripción general
2.1 Perspectiva del producto
2.2 Funciones del producto
2.3 Características del usuario
2.4 Restricciones
2.5 Supuestos y dependencias
2.6 Requisitos pospuestos
3. Requisitos específicos
3.1 Requisitos de la interfaz externa
3.1.1 Interfaz con el usuario
3.1.2 Interfaz con el hardware
3.1.3 Interfaz con el software
3.1.4 Interfaces de comunicaciones
3.2 Requisitos funcionales
3.2.1 Modo 1
3.2.1.1 Requisito 1.1 Funcional
…
3.2.1.n Requisito 1.n Funcional
3.2.2 Modo 2
…
3.2.m Modo m
3.3 Requisitos de rendimiento
3.4 Restricciones de diseño
3.5 Atributos del sistema de software
3.6. Otros requisitos
Apéndices
Índice

A continuación, se detalla que información se debe incluir en cada una de las secciones de esta plantilla.

1. Introducción

La introducción del ERS debe proporcionar una visión global de la ERS completa.

1.1 Propósito

- Propósito de la ERS
- A quién va dirigida

1.2 Alcance

- Identificar el producto software dándole un nombre.
- Explicar que hace el producto, y si es necesario, que no hace.
- Describir su aplicación, indicando beneficios, objetivos y metas.
- Si existen, indicar la consistencia del documento con otras especificaciones de más alto nivel (por ejemplo, especificación de requisitos del sistema)

1.3 Definiciones, siglas, y abreviaturas

- Incluir definiciones de términos, acrónimos y abreviaturas que se necesite conocer su significado para una comprensión adecuada de la ERS. Si es muy extenso, se puede incluir como apéndice.

1.4 Referencias

- Incluir referencias o citas bibliográficas de todos los documentos que se citen en la ERS, o utilizados para su elaboración. Si la bibliografía es muy extensa, se puede incluir como apéndice.

1.5 Visión global

- Descripción del contenido del resto de la ERS y de qué forma está organizada.

2. Descripción general

Esta sección del ERS debe describir los factores generales que afectan el producto y sus requisitos. Esta sección no declara los requisitos específicos. En cambio, mantiene la base común de esos requisitos que se definen en detalle en Sección 3, y les hacen más fáciles de entender.

2.1 Perspectiva del producto

- Definir la relación del producto con otros productos software relacionados.
- Si el producto es totalmente independiente, se debe indicar.
- Si el producto es un componente de un sistema mayor, se debe describir aquí el sistema completo.
- Se deberán de especificar el comportamiento del producto con respecto a:
    - Interfaces del Sistema: Listar y describir cada interfaz del sistema y detallar que funcionalidad software cubriría ese requisito del sistema.
    - Interfaces de Usuario: Especificar:
        - Las características lógicas de cada interfaz entre el producto del software y sus usuarios. Esto incluye las características de la configuración (por ejemplo, formatos de la pantalla requeridos, formatos de páginas o ventanas, informes de menús) necesarias para lograr los requisitos del software.
        - Todos los aspectos para perfeccionar la interfaz para la persona que va a usarla. Esto puede comprender una lista de lo que hace y no hace delante del sistema.
    - Interfaces Hardware: Especificar las características lógicas de cada interfaz entre el producto del software y los componentes hardware del sistema.
    - Interfaces Software: Especificar el uso de otros productos del software requeridos (por ejemplo, un sistema gestor de datos, un sistema operativo o un paquete matemático) e interfaces con otros sistemas de la aplicación.

        - Para cada producto se ha de definir: nombre, código mnemotécnico, número de la especificación, número de la versión, y fuente.
        - Para cada interfaz se ha de definir: propósito y definición de la misma.
    - Interfaces de Comunicaciones: Especificar las interfaces de comunicación, como por ejemplo los protocolos de red.
    - Memoria: Especificar cualquier característica aplicable o límite asociado a la memoria primaria o la memoria secundaria.
    - Operaciones: Especifica las operaciones normales o especiales de los usuarios del sistema software, de inactividad o de copia de seguridad. Esta sección a veces se especifica en las interfaces de usuario.

2.2 Funciones del producto

- Resumen de las principales funciones que el software debe realizar.
- La lista de funciones se deben organizar de forma que queden claras la primera vez que se lee el documento. Si alguna requiere funcionalidades concretas, omitirlas para ganar claridad. Por ejemplo: En un programa de contabilidad en esta parte se describiría el mantenimiento de Cuenta de Cliente, la declaración del cliente y la gestión de facturas, sin mencionar la inmensa cantidad de detalle que cada una de esas funciones requiere.

2.3 Características del usuario

- Describe qué características generales debe tener el usuario que vaya a trabajar en el producto software, en cuanto a experiencia, nivel de conocimientos, formación específica que requiere, etc.

2.4 Restricciones

- Si existen, se mencionan aquellos aspectos que pueden limitar el proceso de desarrollo software en términos de limitaciones hardware, interfaces con otras aplicaciones, políticas reguladoras, aspectos de seguridad, funciones de auditoría, etc.

2.5 Supuestos y dependencias

- Si existen, se mencionan dependencias con otro software, como por ejemplo el sistema operativo, y de qué forma esto puede afectar a los requisitos si dicho software no está disponible.

2.6 Requisitos pospuestos

- Requisitos que se postergan a futuras versiones del sistema.

3. Requisitos específicos

Esta sección es la más extensa del estándar e importante, puesto que contiene todos los requisitos software con un nivel de detalle suficiente como para permitir que los ingenieros de software puedan diseñar un sistema que cumpla todos los requisitos y que se puedan realizar las pruebas que comprueben si se satisfacen dichos requisitos.

3.1 Requisitos de la interfaz externa

Descripción más detallada de las interfaces que completa la sección del estándar 2.1. Para ello se especifican para cada interfaz las entradas y salidas, incluyendo el nombre, descripción, rango de valores válido, tiempo permitido, formato datos, unidades, de medida, formato de pantalla, organización de ventanas, etc. Cada interfaz, atendiendo a su tipología, será descrita en la subsección correspondiente.

3.1.1 Interfaz con el usuario
3.1.2 Interfaz con el hardware
3.1.3 Interfaz con el software
3.1.4 Interfaces de comunicaciones

3.2 Requisitos funcionales

Descripción de las acciones fundamentales, requisitos, que debe realizar el software, es decir, de qué forma se procesan y aceptan cada una de las entradas para producir una determinada salida. En esta plantilla se organizan los requisitos por modos de funcionamiento. Asumiendo que en la asignatura vamos a trabajar en un único modo de funcionamiento, que es el modo normal de uso, no se incluiría esa categorización en la ERS.

3.2.1 Modo 1

3.2.1.1 Requisito 1.1 funcional

Cada requisito funcional debe incluir:

- Descripción: Breve descripción del requisito.
- Entradas: Todas las entradas que necesita
- Procesamiento: El proceso que sigue para transformar las entradas en salidas.
- Salidas: Todas las salidas que produce
- También se puede incluir la respuesta a situaciones excepcionales, errores, fórmulas de conversión de las entradas en salidas, etc.

…

3.2.1.n Requisito 1.n Funcional

3.2.2 Modo 2

…

3.2.m Modo m

3.3 Requisitos de rendimiento

Especificar aquellas prestaciones software que se requieren, tanto *estáticas* como número de terminales soportados, número de usuarios simultáneamente, cantidad de información a ser procesada, como dinámicas como el tiempo de respuesta para ciertas operaciones a realizar por el software

3.4 Restricciones del diseño

Especificar restricciones impuestas por otros estándares o por el hardware

3.5 Atributos del sistema de software

Especificar características a tener en cuenta como fiabilidad, disponibilidad, seguridad, mantenibilidad, o portabilidad.

3.6. Otros requisitos

Apéndices

Índice